



American International University-Bangladesh (AIUB)
Department of Computer Science
Faculty of Science & Technology (FST)

PROJECT TITLE:

PEERSYNC

Semester: Summer 25-26

Group: 05		Section: E
SL	Student Name	Student ID
1	Md. Azraful Alam	24-57476-2
2	Sumaiya Binte Kamal	23-54657-3
3	Mohammad Tahlil Al-Raiyan	24-57417-2
4	Sumaiya Alam Shraboni	24-56174-1
5		

Assessment Rubric

CO1: Explain the impact of software engineering over various contexts of software development to assess societal, health, safety, legal and cultural issues.

Student ID	Problem Analysis [3]	Socio-cultural Impact [3]	Related Solutions and Studies [4]
1.			
2.			
3.			
4.			
5.			

CO2: Explain software engineering principles considering professional responsibilities and societal impacts.

Student ID	Conceptual Understanding [3]	Principle Selection and Justification [3]	Application in Software Development Context [4]
1.			
2.			
3.			
4.			
5.			

Table of Contents

1. Project Proposal	4
1.1 Background to the Problem	4
1.2 Selection of Process Model	5
2. Timeline Chart / Gantt Chart	7
3. Software Requirements Specification (SRS)	8
3.1 Scopes and Features	8
3.2 User Story Table	10
3.3 Requirements Traceability Matrix	21
3.3.1 Functional Requirements	21
3.3.2 Non-Functional Requirements	22
4. Software Design	24
4.1 System Design	24
4.2 UI / Wireframe Design using Figma	24
5. Git Workflow	26
Raiyan	27
Sumaiya	29
Azraf	30
Shraboni	31
Git Merge	32
Final Trello Board	34
6. Conclusion	35

1. Project Proposal

1.1 Background to the Problem

Group-based coursework, capstone projects, and collaborative research are central to how universities operate, yet finding the right collaborator remains largely a matter of luck. Students typically default to whoever they already know — the same three or four friends from a previous semester, or whoever happens to sit nearby — rather than choosing teammates based on complementary skills, compatible working styles, or shared availability. Instructor-assigned groups solve the fairness problem on paper but often make it worse in practice, pairing students with wildly different work ethics, schedules, or skill levels with no mechanism to signal fit beforehand. The result, semester after semester, is a familiar pattern: uneven workload distribution, communication breakdowns, missed deadlines, and project quality that reflects the group's ability to coordinate more than the members' actual competence.

This is not a problem general-purpose tools were built to solve. Facebook groups, WhatsApp, and Discord servers are excellent at letting people who already know each other communicate, but they offer no way to discover someone new, no way to verify that a stranger is actually enrolled at a legitimate institution, and no structured way to advertise what skills or working style a potential collaborator brings to the table. Once a project ends, there's also no lasting record of how well someone actually performed as a teammate — good collaborators and poor ones look identical to the next group that inherits them, and reputations built through hard work simply evaporate.

PeerSync began as a direct response to this gap at AIUB. The original version used a swipe-based matching model — similar in spirit to consumer dating apps, but built around academic compatibility instead — to help students find teammates for coursework based on complementary skills and comparable working styles. Early use validated the core idea: students genuinely preferred discovering teammates through structured matching over defaulting to their existing social circle, and a lightweight post-project rating system gave good collaborators a way to build a visible track record. But that first version was narrow by design, limited to a single institution and a single use case — group formation for individual class projects — which meant it addressed only a slice of a much larger problem.

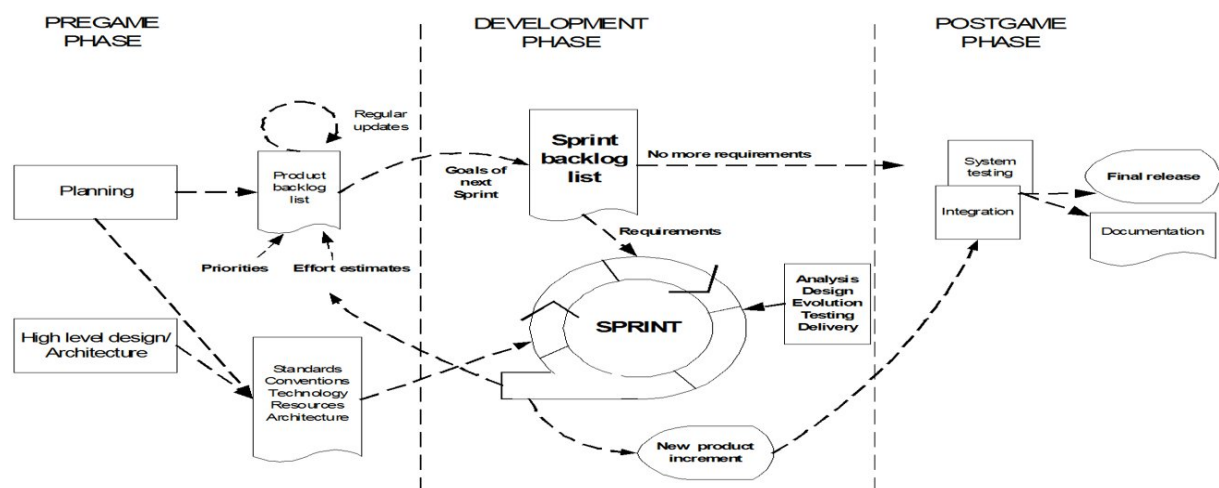
The difficulty of finding the right collaborator is not unique to one university, one class, or even to students. Professors and researchers face an almost identical challenge when looking for collaborators on cross-departmental or inter-university research — discovery still happens mostly through personal networks and conference hallway conversations, rather than through any structured channel. Mentors and teaching assistants, similarly, have no dedicated way to offer guidance to students outside their own assigned class list, even when their expertise would be directly relevant. Recognizing that the underlying problem — trustworthy, skill-based discovery of academic collaborators — scales well beyond a single AIUB classroom, PeerSync is being expanded into a multi-university academic collaboration platform.

The redesigned platform keeps the core insight that made the original version work — structured, skill-based matching beats chance-based grouping — while addressing the trust and scale problems that a single-university tool didn't have to solve. Verified institutional email accounts establish that every user is a real, currently-enrolled member of a legitimate university, extending the trust that used to come for free from being on one campus. Intelligent search and filtering let a student, professor, or mentor surface a compatible collaborator by skill, department, or research interest, rather than relying on chance encounters or existing

social ties. Built-in real-time messaging keeps the entire collaboration lifecycle — from first contact through project completion — inside one accountable system, and a persistent reputation score means that a good collaborator's track record now follows them across every project and every institution they work with, rather than resetting every semester. Together, these give students, faculty, and researchers across universities a shared, trustworthy space to find the right academic partner — and give every stakeholder involved in the project a common view of exactly what problem PeerSync is meant to solve.

1.2 Selection of Process Model

We chose the Agile Scrum model for the PeerSync project due to its flexibility, iterative development approach, and strong support for changing requirements. As the system involves multiple user roles such as students, professors, researchers, industry partners, and administrators, their needs and requirements may evolve during development. Scrum's sprint-based approach allows the team to deliver working features incrementally, enabling early feedback and continuous improvement. Key functionalities such as user registration and verification, profile management, collaborator search, project posting, messaging, research collaboration, and administrative features can be developed, tested, and refined across multiple sprints. Scrum also promotes effective communication through regular ceremonies such as Sprint Planning, Daily Scrum, Sprint Reviews, and Sprint Retrospectives, which is suitable for our team working within limited academic time and resources.



We did not use other models due to:

Waterfall Model: The Waterfall model follows a rigid, sequential approach with fixed requirements. It was not chosen because PeerSync contains multiple user roles and numerous features, and requirements may change based on user and supervisor feedback. Its limited support for early feedback makes it less suitable for the dynamic development of PeerSync.

V-Model: The V-Model emphasizes verification and validation and requires extensive upfront planning and documentation. This rigidity makes it less suitable for PeerSync, where features can be improved through continuous feedback and iterative development.

Extreme Programming (XP): XP focuses heavily on coding practices such as pair programming, test-driven development, continuous integration, and refactoring. However, PeerSync requires stronger project coordination and management of many user stories and

user roles. Scrum provides more structured sprint planning and progress monitoring, making it more suitable for this project.

Dynamic Systems Development Method (DSDM): DSDM provides strong governance and prioritization but involves additional roles, processes, and documentation. This overhead is not practical for a small academic team working with limited time and resources.

Feature Driven Development (FDD): FDD is more appropriate for larger development teams and complex systems with specialized feature ownership. Its formal feature-driven structure is less suitable for PeerSync, where flexible teamwork and continuous feedback are important.

Scrum is an agile framework that delivers software incrementally through time-boxed sprints, allowing continuous feedback and adaptation. For the PeerSync project, Scrum provides an effective balance between structure and flexibility, enabling the team to prioritize important features early while continuously developing, testing, and improving the system based on user and stakeholder feedback. This makes Scrum suitable for developing a flexible and high-quality multi-university academic collaboration platform.

2. Timeline Chart / Gantt Chart

Phase	Task Description	Priority	Duration	Start Date	End Date	Responsible
Phase 1: Planning	Project Proposal, Requirement Gathering & Team Role Assignment	–	1 Week	May 15, 2026	May 21, 2026	All Members
Phase 2: Analysis	SRS Documentation & User Story Mapping (Trello Backlog Setup)	–	1 Week	May 22, 2026	May 28, 2026	All Members
Phase 3: Design	System Architecture, Database Design, UML & Wireframes	–	2 Weeks	May 29, 2026	Jun 11, 2026	Shraboni, Raiyan
Phase 4: Sprint 1	Admin & Student Core Modules (Login, Verification, Profile, Collaborator Search, Team Chat)	High	2 Weeks	Jun 12, 2026	Jun 25, 2026	Raiyan
Phase 5: Sprint 2	Professor & Researcher Core Modules (Verification, Research Lab, ORCID Sync, Partner Search)	High	2 Weeks	Jun 26, 2026	Jul 09, 2026	Azraf
Phase 6: Sprint 3	Industry Partner & Remaining Modules (Corporate Registration, Talent Search, Chat, Vacancies)	High	2 Weeks	Jul 10, 2026	Jul 23, 2026	Sumaiya
Phase 7: Sprint 4	Analytics, Announcements, Leaderboard, Notifications, Messaging & Remaining Features	Medium / Low	2 Weeks	Jul 24, 2026	Aug 06, 2026	All Members
Phase 8: Testing	Manual Test Cases, Bug Fixing & QA Review	–	2 Weeks	Aug 07, 2026	Aug 20, 2026	Sumaiya, Azraf
Phase 9: Review & Delivery	Documentation, Final Report Preparation & Delivery	–	1 Week	Aug 21, 2026	Aug 27, 2026	Shraboni

3. Software Requirements Specification (SRS)

3.1 Scopes and Features

1. User Registration and Authentication

- Multi-role registration: students, professors, researchers, industry partners, administrators
- Registration using official institutional or corporate email verification
- Secure login and authentication
- Account verification and approval for professors, researchers, and industry partners
- Password recovery and account security
- Session management and role-based access control

2. Profile Management

- View and update personal and academic information
- Students: university, major, completed courses, technical skills, and hackathon status
- Professors: institution, research interests, publications, and thesis opportunities
- Researchers: research field, publications, skills, and academic identifiers
- Industry partners: organization information, corporate verification, and areas of interest
- Privacy controls for protecting personal and institutional information

3. Collaborator Search and Discovery

- Search users across multiple universities and institutions
- Filter profiles by university, major, skills, course codes, research interests, and other criteria
- Students can find suitable academic collaborators based on skills and requirements
- Professors can search for students and faculty members based on expertise
- Researchers can find cross-disciplinary and cross-institutional collaborators
- Industry partners can search for students based on skills and reputation

4. Project and Team Collaboration

- Create and publish project or team vacancy posts
- Students can request to join available project teams
- Professors can publish thesis and laboratory assistant opportunities
- Researchers can create private research project workspaces
- Industry partners can publish innovation challenges, hackathons, and capstone topics
- Manage project members and collaboration activities

5. Messaging and Communication

- Direct messaging between verified users
- Group messaging for project and research teams
- Real-time project communication

- Message history and member presence indicators
- Share repository links, cloud folders, and project resources through quick actions
- Secure communication for research and industry collaboration

6. Research and Academic Collaboration

- Professors can publish research themes, thesis positions, and laboratory opportunities
- Researchers can synchronize publication information using ORCID or Google Scholar identifiers
- Create private research workspaces and project groups
- Upload and share research papers, datasets, manuscripts, and other resources
- Invite professors and researchers to research or grant collaboration groups
- Manage project milestones and research deadlines

7. Reputation, Rating and Leaderboard System

- Multi-dimensional peer rating system
- Students can rate teammates based on contribution, communication, and reliability
- Automatic calculation and updating of user Karma scores
- Display global and university-based leaderboards
- Industry partners can provide industry-endorsed performance scores
- Verified reputation information helps users identify reliable collaborators

8. Notifications and Activity Management

- Real-time notifications for project requests and invitations
- Notifications for messages and collaboration activities
- Research-related alerts based on saved interests
- Notifications for project milestones and deadlines
- Industry partners can receive alerts when relevant student projects are published
- System-wide announcements from administrators

9. Privacy, Security and Verification

- Institutional and corporate identity verification
- Role-based access control
- Secure authentication for administrator accounts
- Privacy settings for personal and university information
- Restricted access to private research and project workspaces
- Secure document and resource sharing
- Administrative moderation and user-report management

10. Administration and Platform Management

- Secure administrator dashboard with multi-factor authentication

- Manage and verify user accounts
- Manage institutional email domain whitelist
- Review reported users and system activities
- Remove fraudulent or reported profiles
- Monitor platform analytics and user activity
- Broadcast system-wide announcements
- Enable maintenance mode when required

3.2 User Story Table

User stories are grouped by role. Each row captures a single story with its acceptance criteria in Given / When / Then format.

Admin (8 stories)

I want to	So that	Acceptance Criteria
Access a centralized platform configuration panel via a secure multi-factor authentication login.	I can execute administrative commands and ensure that malicious external actors cannot gain elevated system access.	Description: Secure authentication protocol for administrative staff accounts. Priority: High Given: The user is attempting to access the /admin route pathway. When: The user inputs valid admin credentials and passes a secondary code challenge. Then: The platform provisions session state permissions and renders the administrative master dashboard framework.
Delete a reported or fraudulent profile from the system console using background cascade execution logic.	I can completely purge bad users without causing foreign key execution errors or leaving orphaned records in auxiliary lookup tables.	Description: Administrative data clean-up utilizing sequential background transactions across child tables. Priority: High Given: The Admin clicks the "Delete User" action command button on a specific user profile item. When: The database system initiates the execution context sequence. Then: It utilizes a transaction scope to drop matches, group assignments, skill vectors, ratings, and preferences sequentially before safely removing the core user registration identity entry.
Manage and append valid institutional domain patterns onto the system registration whitelist tracker.	The application can automatically scale up or down to authorize access to newly partnered regional universities.	Description: Direct CRUD workspace panel tracking valid university email criteria rules. Priority: High Given: The Admin opens the institutional email whitelist config board. When: The Admin types a new domain string (e.g., @du.ac.bd) and clicks "Append Domain Entry."

		Then: The database updates the verification lookup tables instantly, allowing students from that campus to register.
View a centralized feed log listing active user reports and flagged system item interactions.	I can investigate toxic behavioral patterns, review false accounts, and maintain platform security standards.	Description: Real-time log monitoring dashboard aggregating system-wide user flag events. Priority: High Given: A user submits a violation report card against a shared project post. When: The system processes the event flag string parameter. Then: The entry populates the top of the Admin's moderation queue window, containing timestamps, user IDs, and conflict context.
Review profile applications and submitted verification files from newly registered Professors and Industry Collaborators.	I can manually authorize their special system badges and confirm they are official representatives before they interact with students.	Description: Review and manual approval portal for user accounts demanding elevated platform trust markers. Priority: High Given: A faculty member completes their advanced profile authorization workflow registration forms. When: The Admin clicks the "Approve Academic Identity" checkbox inside the verification panel. Then: The system changes the target user status token to "Verified" and assigns their permanent academic badge.
Generate global platform metrics displaying active registration counts, matching success rates, and cross-university project numbers.	I can review platform performance trends and compile usage analytics reports for academic stakeholders.	Description: Analytical chart query aggregator loading data variables onto a visual monitoring grid. Priority: Medium Given: The Admin clicks onto the platform "Analytics Hub" sidebar panel component. When: The page executes background counting loops on the relational data infrastructure tables. Then: The UI generates comparative line charts displaying engagement volumes sorted by institution.
Inject systemic platform broadcast banners directly across all active user timeline view headers.	I can quickly announce maintenance downtime schedules, platform policy transformations, or regional hackathon events.	Description: System-wide content broadcast execution utility. Priority: Medium Given: The Admin drafts an urgent maintenance brief in the system messaging console. When: The Admin clicks the "Execute System-Wide Broadcast" action button. Then: A banner component renders across the UI headers of all currently connected accounts in real-time.

Trigger a "Global Maintenance Mode" state toggle on the platform.	I can temporarily restrict public interactions and perform data schema updates or structural index repairs safely.	Description: Creative system-level feature providing an emergency access gate to block incoming public API requests during deployments. Priority: Low Given: The platform is experiencing a critical database infrastructure failure. When: The Admin enables the master system maintenance toggle switch indicator. Then: All non-admin user requests are intercepted and directed to an static "Under Maintenance" placeholder screen view.
---	--	---

Student (10 stories)

I want to	So that	Acceptance Criteria
Register on the platform using my official institutional university email address.	I can securely join my university's campus network and prevent unverified public accounts from accessing our space.	Description: Validates user registration against a whitelist of academic email domains. Priority: High Given: The student is on the registration page. When: The student inputs a valid email string (e.g., @aiub.edu, @g.bracu.ac.bd) and submits. Then: The system updates account status to "Pending Verification" and triggers an email dispatch containing a verification link.
Create and customize my academic profile with my major, completed course history, and skill tags.	Other project creators across universities can explicitly search for me based on my competencies.	Description: Allows students to showcase their current courses and technical skills (e.g., C++, Java, Python, JavaScript, CSS). Priority: High Given: The student has verified their account and is logged in. When: The student selects technical skills from a dynamic tag list and enters their major. Then: The system appends these details to their public card view and indexes them in the search registry.
Access an advanced search dashboard equipped with fields for University, Major, Skills, and Course Codes.	I can explicitly locate a precise peer match for an assignment instead of relying on random profile swiping.	Description: A search tool with strict text and category filter combinations. Priority: High Given: The student is on the "Collaborator Discovery Hub." When: The student filters by "AIUB", selects the skill "C#", and searches for course code "CSE3103". Then: The UI updates its grid system in real-time to display only student profiles

		satisfying all matching metrics.
Publish a detailed team vacancy listing card onto a centralized project board.	Qualified students from different universities can see our vacancy and apply to join our assignment group.	Description: Form-based posting framework for the platform's public Project Collaboration Feed. Priority: High Given: The student is managing an open group assignment. When: The student defines the project goal, inputs a required skill slot ("Frontend Developer"), and hits publish. Then: A vacancy card populates the public feed displaying a 1-click "Request to Join" button.
Launch a dedicated text messaging canvas with my approved project teammates.	We can collaborate and coordinate milestones directly within the platform without moving to external applications.	Description: Real-time text interaction workflow within the communication hub. Priority: High Given: A student has accepted a peer's incoming group application request. When: The student selects the team name from their active side roster pane. Then: A secure messaging chat box opens showing historical text entries and member presence indicators.
Click a "+" quick-actions panel within my active team chat stream.	I can instantly insert interactive links to shared cloud folders or code repositories without leaving our text discussion.	Description: Creative flyout menu shortcut integration inside message input spaces. Priority: Medium Given: The student is typing inside an active project group chat window. When: The student clicks the "+" button and selects the "Drop Repository Link" action option. Then: A styled structural link card displaying repository details is rendered directly into the chat stream for all group members.
Open the platform's global or campus leaderboard interface.	I can discover the highest-rated student collaborators and check my own cumulative karma position.	Description: Gamified ranking interface ordered by community reputation metrics. Priority: Medium Given: The student clicks on the platform "Leaderboard" tab. When: The student switches the view toggle from "Global Network" to "My University". Then: The system generates a sorted table layout ordered descending by user Karma scores, appending visual status borders (Gold, Silver, Bronze) to top profiles.
Complete a multi-vector slider rating form for my group members when a project wraps up.	I can accurately log peer performance scores to maintain strict network accountability.	Description: Peer review evaluation system rating teammates across contribution, communication, and reliability.

		Priority: High Given: A project manager flags an assignment milestone as complete. When: The student fills out performance metric sliders from 1 to 5 for a teammate and clicks submit. Then: The system updates the database records and mathematically recalculates the target peer's public Karma score.
Configure granular privacy toggles within my account security dashboard.	I can hide my personal email text string and university ID numbers from users who are not active team members.	Description: Privacy and permission restriction management to protect sensitive student identifiers from data harvesting. Priority: High Given: An unlinked student is browsing user profile cards. When: The profile page renders details. Then: The email and registration digits are replaced with an obfuscated text placeholder saying "Connect to unlock contact info."
Enable an "Open for Inter-University Hackathons" status flag on my profile settings.	External squads from other institutions can discover my profile and invite me to complete their competitive teams.	Description: Creative feature promoting cross-campus innovation and extra-curricular project team assembly. Priority: Medium Given: The student is configuring their profile search parameters. When: The student toggles the "Hackathon Recruitment" status switch to active. Then: A distinct badge asset appends to their public profile container, and they are indexed under the global competition team database view.

Professor (9 stories)

I want to	So that	Acceptance Criteria
Register an identity profile using my verified faculty domain credentials.	I can claim my official academic authority tier on the platform and build a secure mentorship space.	Description: Administrative onboarding workflow designed strictly for active university teaching faculty. Priority: High Given: The instructor is on the specialized onboarding registration page. When: The user enters a recognized faculty domain address and inputs an institution badge verification token. Then: The account upgrades to an active profile embedded with a permanent "Verified Professor" indicator label.
Catalog my active research themes, publications, and open thesis lines on my public profile.	External researchers and students can easily align with my work and apply for mentorship under my guidance.	Description: Profile curation panel focusing on scholastic achievements and instructional pathways.

		Priority: High Given: The professor is modifying their active profile dashboard space. When: The user inputs text parameters under the "Research Interests" component window and hits save. Then: These fields render instantly as searchable hyperlinked filter tags on their public-facing workspace header.
Run a lookup search through a directory of faculty members at other academic institutions.	I can connect with domain specialists to co-author papers or establish cross-university thesis panels.	Description: Advanced multi-institutional specialist discovery engine. Priority: High Given: The professor is logged into the "Academic Network Directory." When: The professor types "Machine Learning" into the search field and narrows the university filter dropdown to a different institution. Then: The directory returns a filtered list table of verified faculty matching those domain parameters.
Publish formal thesis positions or laboratory assistant vacancies to the student feed.	I can collect application profiles from qualified student candidates across different campuses.	Description: Structural vacancy posting engine for academic research lines. Priority: High Given: The professor opens the "Academic Placement Board." When: The professor drafts a thesis project outline and sets a minimum skill prerequisite requirement. Then: The platform distributes the post to the student feed, limiting applications to students who meet the academic criteria.
Create an insulated virtual research laboratory project container.	I can track my research assistants, group chats, and draft paper documents in one place.	Description: Dedicated sub-group workspace provisioning for institutional labs. Priority: High Given: The professor needs to track a new research team cycle. When: The professor inputs a laboratory title string and clicks "Launch Lab Container." Then: A secure dashboard is established, generating a member list table window to invite selected researchers.
Initiate a direct message dialogue with a student directly from the campus leaderboard view.	I can recruit high-performing student talent for my funded R&D projects.	Description: Targeted recruitment communication route bypassing standard matching workflows. Priority: Medium Given: The professor is evaluating student profiles on the public Karma Leaderboard. When: The professor selects a top-tier candidate card and clicks "Send

		Mentorship Invitation." Then: A direct text window opens inside the student's message layout featuring an official faculty verification header flag.
Access a secure resource dashboard within my virtual laboratory group.	I can upload and distribute draft manuscripts, data tables, and syllabi to my research assistants safely.	Description: Restricted document repository management module for team workspaces. Priority: High Given: The professor is within their private Research Group window. When: The professor drags a document file onto the workspace asset dashboard zone. Then: The system saves the resource, updates the shared repository list, and appends an author timestamp log.
Toggle on automated notifications for global academic conferences and symposium calls.	I am automatically alerted about relevant paper submission deadlines that align with my profile tags.	Description: Context-aware event discovery notification manager parsing profile keyword values. Priority: Medium Given: An external call for papers matching "IoT Logistics" is posted to the platform. When: The system parses the professor's saved research tags. Then: An alert card populates the professor's primary dashboard notification panel highlighting the submission deadline.
Request a "Blind Student Match Recommendation" from the platform discovery hub.	The system can suggest qualified candidates based purely on verified skill metrics while removing demographic bias.	Description: Creative bias-reduction recruitment feature anonymizing student identifier strings during selection. Priority: Medium Given: The professor needs an assistant with explicit technical skills. When: The professor chooses the "Blind Match" tool option and sets parameters for "Python" and "Data Analysis". Then: The interface presents a list table of matching student profiles with names and IDs hidden, exposing only skill graphs and peer ratings.

Researcher (8 stories)

I want to	So that	Acceptance Criteria
Sync my PeerSync profile layout with my official external ORCID or Google Scholar identifier string.	My verified publication records and citation history display automatically on my platform portfolio card.	Description: Identity profile syncing utilizing public academic metadata indexes. Priority: High Given: The researcher is setting up their professional account profile. When: The researcher enters an ORCID digit id and clicks the "Sync

		Publications" request button. Then: The system reads the public metadata registry and updates their profile view with their verified publication count.
Search the directory using complex keyword filter combinations like primary field plus technical skill.	I can find cross-domain technical partners for joint interdisciplinary projects.	Description: Granular cross-institutional search matrix engine. Priority: High Given: The researcher needs a technical partner outside their discipline. When: The user applies a keyword parameter combination for "Bioinformatics" AND "Python Programming." Then: The list matrix grid matches the constraints and returns only profiles that hold both skill indicators.
Launch an encrypted "Project Sandbox Vault" with customized user access permissions.	Our core team can brainstorm patent ideas or draft paper outlines while protecting our intellectual property.	Description: High-security private project workspace provisioning. Priority: High Given: The researcher is initiating a high-security research project. When: The researcher inputs a title and sets the group directory visibility toggle state to "Strictly Private." Then: The project database restricts entry visibility to authorized team member accounts.
Upload open-source datasets or preprint papers onto a shared public resource feed.	I can gather early citations and secure peer validation feedback from the regional academic community.	Description: Public-facing repository board supporting asset dissemination. Priority: Medium Given: The researcher is on the platform's public Resource Board. When: The researcher uploads an asset and sets its license type selection field to "Open Access." Then: The document populates the main community timeline accompanied by an integrated reference generator tool.
Send a workspace participation invite link to verified senior university faculty.	We can collaborate on grant-writing proposals within a shared, private document group.	Description: Collaboration onboarding link workflow bridging independent researchers and faculty staff. Priority: High Given: The researcher is building a grant workspace group. When: The researcher selects a professor profile from the user directory and clicks "Invite to Grant Group." Then: An invitation notification package containing project outline summaries is sent to the target professor's inbox.
Upload and share raw file data formats directly inside our team message feed.	Team members can instantly download and run code adaptations during research	Description: Secure drag-and-drop file asset allocation matching chat canvas interfaces.

	cycles.	Priority: Medium Given: The researcher is corresponding within their project team chat panel. When: The researcher drops an updated file onto the text entry zone container. Then: The asset renders instantly into the conversation flow as an interactive downloadable attachment module.
Configure a chronological milestone timeline on our project dashboard.	Our distributed team can align on upcoming journal submission deadlines and draft reviews.	Description: Light project roadmap management timeline mapping targeted goals. Priority: Medium Given: The researcher is viewing their project's administrative sub-view. When: The user schedules a new target goal ("Complete First Draft") and defines a calendar date. Then: The milestone maps to the shared team dashboard and pushes alert cards to all active member accounts.
Post a "Research Crowdsourcing Request" detailing data collection fields and survey links to student timelines.	I can gather survey responses from verified university students across multiple campuses.	Description: Creative feature that allows researchers to safely tap into the platform's student user base for studies. Priority: Medium Given: The researcher requires data samples for an active research study. When: The researcher fills out a "Crowdsourcing Call" form with an external survey link and hits publish. Then: A specialized call widget populates the global student dashboard stream, tracking participation metrics in real-time.

Industry Partner (9 stories)

I want to	So that	Acceptance Criteria
Register an account utilizing my corporate email domain.	I can represent my organization in university spaces and display an official industry verification marker.	Description: Corporate registration onboarding checking company email strings to ensure network trust. Priority: High Given: The collaborator is initializing a corporate account profile. When: The user enters an enterprise email string (e.g., @microsoft.com) and confirms verification. Then: The account provision script activates the workspace and flags all their posts with an official corporate badge.
Publish sponsored innovation challenges, hackathons, or capstone topics onto a global project feed.	University laboratories and student teams can review our challenge requirements and pitch their engineering solutions.	Description: Innovation sponsorship project board posting functionality for corporate actors. Priority: High

		Given: The corporate partner is inside their "Sponsorship Challenge Hub." When: The user outlines a challenge brief, inputs R&D funding details, and submits the form. Then: The challenge goes live on a global Innovation feed accessible to all verified professor and student groups.
Search the student network using filter sliders for precise Majors, technical Skills, and verified Karma tiers.	Our corporate recruitment team can discover high-performing students for specialized internships.	Description: Filter-driven talent-pool acquisition search model scanning student performance matrices. Priority: High Given: The industry recruiter opens the platform "Talent Discovery Hub." When: The recruiter applies filters for "Software Engineering," a high karma rank tier, and the skill tag "C++". Then: The directory lists matching student profiles sorted by their peer-reviewed teamwork rating.
Establish an encrypted private dialogue thread with university laboratory heads.	We can discuss technology licensing, patent applications, and corporate commercialization options securely.	Description: Protected, flagged communication path for intellectual property consulting. Priority: High Given: The industry collaborator visits a professor's research profile canvas. When: The user clicks the "Initiate Corporate Advisory Thread" button tool. Then: A direct chat canvas launches, featuring a persistent non-disclosure privacy warning label across the interface footer.
Distribute a multi-institutional workshop proposal notification to multiple departments at once.	We can organize inter-campus engineering workshops or joint research consortiums.	Description: Mass broadcast proposal distribution tool targeting specific faculty categories. Priority: Medium Given: The corporate user is drafting an engineering consortium call. When: The user writes the scope brief and ticks checkboxes for "AIUB," "BRACU," and "DU." Then: The platform dispatches individual notification requests directly to the faculty dashboards in those chosen departments.
Embed corporate requirement document packages directly inside my active project chat spaces via quick-actions.	University student teams assigned to our sponsored challenges can easily review our technical guidelines.	Description: Quick-action chat link card deployment for enterprise asset documentation. Priority: Medium Given: The corporate partner is chatting within an active challenge workspace room. When: The user logs an asset via the "+" menu flyout selecting "Link Corporate

		Guideline Pack." Then: An interactive file container downloads block is dropped directly into the message text stream.
Set up dynamic alert rules tracking targeted student capstone tags.	I receive a dashboard notification the moment a student project group logs their final prototype as "Published."	Description: Automated push notification rule engine matching corporate interest parameters with published outputs. Priority: Medium Given: A student team sets their final capstone project state to "Published" with the tag "Machine Learning." When: The corporate user has "Machine Learning" listed in their monitored tracking interests. Then: An alert card populates the partner's dashboard feed linking straight to the project's public abstract.
Fill out a post-project industrial readiness score form for student teams completing a challenge.	I can append an official industry-endorsed verification metric to their public career profiles.	Description: Post-project evaluation system allowing sponsors to validate student corporate readiness. Priority: Medium Given: A sponsored innovation challenge timeline reaches its completion date. When: The corporate sponsor submits their final student performance assessment scores. Then: The platform adds a distinct "Industry Endorsed" score badge asset to the student's profile card layout.
Launch a virtual "Pitch Day Showcase Lounge" invitation event within the platform.	Selected academic research teams can pitch their startup prototypes directly to our venture funding group.	Description: Creative feature providing an interactive portal within PeerSync where industry sponsors can host digital product pitches. Priority: Low Given: The corporate partner wants to evaluate active project teams for venture funding. When: The partner clicks "Schedule Pitch Day" and selects matching student groups from their checklist dashboard. Then: The system creates a virtual showcase lounge view and issues calendar alert notifications to all selected teams.

3.3 Requirements Traceability Matrix

3.3.1 Functional Requirements

ID	Functional Requirement	Role	Priority
FR-01	The system shall allow students to register using their institutional university email address.	Student	High
FR-02	The system shall allow students to create and customize an academic profile with major, course history, and skill tags.	Student	High
FR-03	The system shall provide an advanced search dashboard filterable by University, Major, Skills, and Course Codes.	Student	High
FR-04	The system shall allow students to publish team vacancy listings to a centralized project board.	Student	High
FR-05	The system shall provide real-time text messaging between approved project teammates.	Student	High
FR-06	The system shall allow students to insert links to shared repositories/cloud folders via a quick-action menu inside chat.	Student	Medium
FR-07	The system shall display a global and campus-level leaderboard ranked by Karma score.	Student	Medium
FR-08	The system shall allow students to submit multi-criteria peer ratings for teammates upon project completion.	Student	High
FR-09	The system shall allow students to configure privacy toggles that hide personal contact details from non-team members.	Student	High
FR-10	The system shall allow students to enable an “Open for Inter-University Hackathons” status flag on their profile.	Student	Medium
FR-11	The system shall allow administrators to log in via a secure multi-factor authentication panel.	Admin	High
FR-12	The system shall allow administrators to permanently delete fraudulent or reported profiles using cascading background deletion.	Admin	High
FR-13	The system shall allow administrators to manage a whitelist of valid institutional email domains.	Admin	High
FR-14	The system shall provide administrators a centralized moderation feed of user reports and flagged interactions.	Admin	High
FR-15	The system shall allow administrators to review and approve verification submissions from Professors and Industry Collaborators.	Admin	High
FR-16	The system shall generate platform-wide analytics on registrations, matching rates, and cross-university project counts.	Admin	Medium
FR-17	The system shall allow administrators to broadcast announcement banners across all active user sessions.	Admin	Medium
FR-18	The system shall allow administrators to toggle a Global Maintenance Mode restricting non-admin access.	Admin	Low
FR-19	The system shall allow professors to register using a verified faculty domain and institution badge token.	Professor	High
FR-20	The system shall allow professors to catalog research themes, publications, and open thesis lines on their profile.	Professor	High
FR-21	The system shall provide a directory search of faculty members across partner institutions.	Professor	High
FR-22	The system shall allow professors to publish thesis or lab-assistant vacancies to the student feed.	Professor	High
FR-23	The system shall allow professors to create an insulated virtual	Professor	High

	research laboratory workspace.		
FR-24	The system shall allow professors to initiate direct messages with students from the leaderboard view.	Professor	Medium
FR-25	The system shall provide a restricted resource dashboard for uploading/distributing documents within a research group.	Professor	High
FR-26	The system shall notify professors of academic conferences and calls-for-papers matching their saved research tags.	Professor	Medium
FR-27	The system shall provide a “Blind Match” recommendation tool that anonymizes student identity while matching on skill metrics.	Professor	Medium
FR-28	The system shall allow researchers to sync their profile with an external ORCID or Google Scholar identifier.	Researcher	High
FR-29	The system shall support complex keyword-combination search across the researcher directory (e.g., field + skill).	Researcher	High
FR-30	The system shall allow researchers to launch an encrypted “Project Sandbox Vault” with configurable access permissions.	Researcher	High
FR-31	The system shall allow researchers to upload open-source datasets/preprints to a shared public resource feed.	Researcher	Medium
FR-32	The system shall allow researchers to send workspace participation invites to verified faculty.	Researcher	High
FR-33	The system shall allow file uploads directly inside the team message feed.	Researcher	Medium
FR-34	The system shall allow researchers to configure a chronological milestone timeline on the project dashboard.	Researcher	Medium
FR-35	The system shall allow researchers to post “Research Crowdsourcing” requests with survey links to student timelines.	Researcher	Medium
FR-36	The system shall allow industry partners to register using a verified corporate email domain.	Industry Partner	High
FR-37	The system shall allow industry partners to publish sponsored innovation challenges/hackathons to a global feed.	Industry Partner	High
FR-38	The system shall provide filtered talent-search of students by Major, Skills, and Karma tier.	Industry Partner	High
FR-39	The system shall allow industry partners to establish encrypted private dialogue threads with faculty.	Industry Partner	High
FR-40	The system shall allow industry partners to distribute multi-institution workshop proposal notifications.	Industry Partner	Medium
FR-41	The system shall allow industry partners to embed corporate requirement document packages inside active project chats via quick-actions.	Industry Partner	Medium
FR-42	The system shall allow industry partners to set up dynamic alert rules tracking targeted capstone project tags.	Industry Partner	Medium
FR-43	The system shall allow industry sponsors to submit a post-project industrial readiness score for student teams.	Industry Partner	Medium
FR-44	The system shall allow industry partners to launch a virtual “Pitch Day Showcase” invitation event.	Industry Partner	Low

3.3.2 Non-Functional Requirements

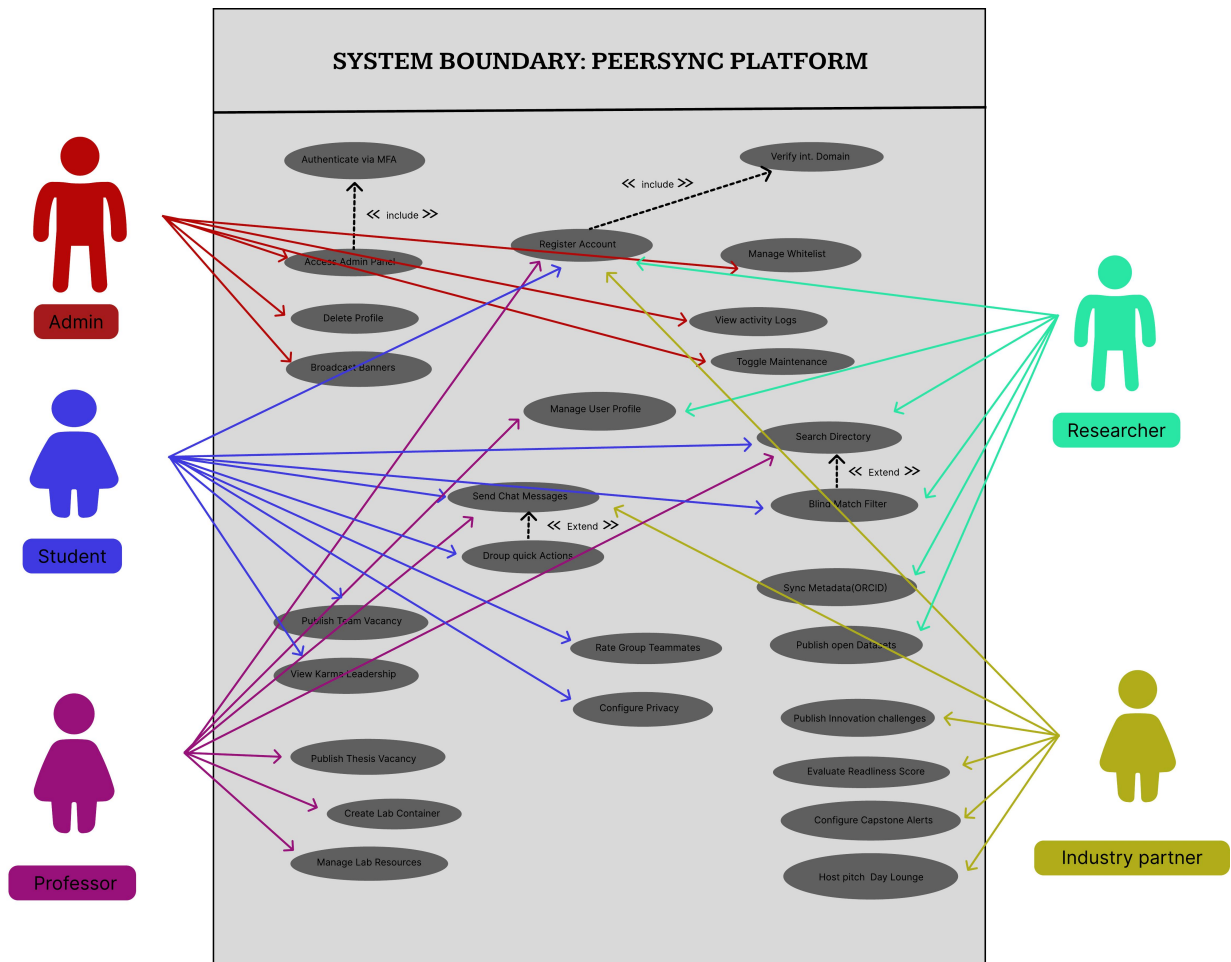
ID	Non-Functional Requirement	Quality Attribute	Related Role	Priority
NFR-01	The system shall load the student dashboard within 3 seconds under normal usage.	Performance	Student	High

NFR-02	The system shall encrypt all private messaging and vault data both in transit and at rest.	Security	Student / Researcher	High
NFR-03	The system shall enforce email-domain verification before granting role-based access (Student/Professor/Industry).	Security	Admin	High
NFR-04	The system shall support concurrent access from users across multiple partner universities without performance degradation.	Scalability	All Roles	High
NFR-05	The system shall update search/filter results in real time (under 1 second) as filters are applied.	Performance	Student	High
NFR-06	The system shall mask sensitive identifiers (email, university ID) from unauthorized viewers by default.	Privacy	Student	High
NFR-07	The system shall maintain 99.5% uptime excluding scheduled maintenance windows.	Reliability	Admin	High
NFR-08	The system shall log all administrative actions (deletions, approvals, broadcasts) for audit purposes.	Auditability	Admin	Medium
NFR-09	The system shall present a consistent, intuitive UI across student, professor, researcher, and industry role views.	Usability	All Roles	Medium
NFR-10	The system shall degrade gracefully into a maintenance placeholder screen without data loss during Maintenance Mode.	Reliability	Admin	Medium
NFR-11	The system shall support file uploads up to a defined size limit without blocking the chat interface.	Performance	Researcher	Medium
NFR-12	The system shall anonymize identity fields in “Blind Match” results to prevent demographic bias.	Fairness / Privacy	Professor	Medium
NFR-13	The system shall be accessible via modern web browsers without requiring additional plugins.	Compatibility	All Roles	Medium
NFR-14	The system shall recalculate and persist Karma scores within a bounded time after peer-review submission.	Performance	Student	Medium
NFR-15	The system shall retain moderation and report logs for a minimum defined retention period for compliance review.	Compliance	Admin	Low

4. Software Design

4.1 System Design

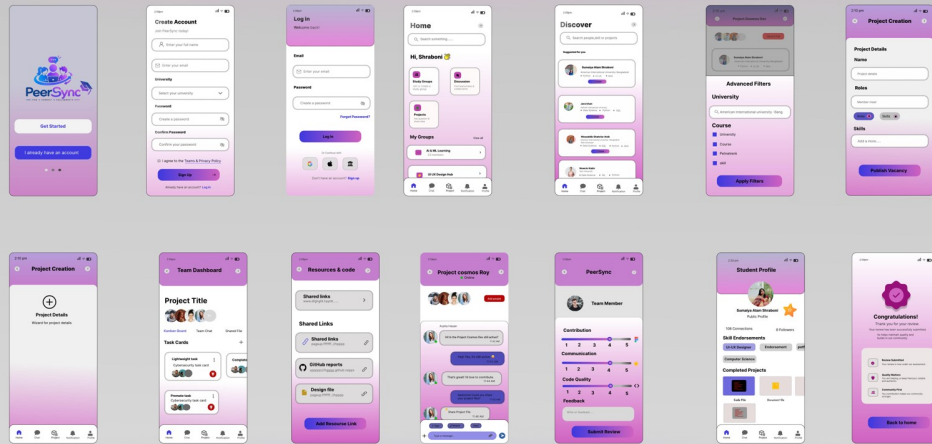
The use case diagram below defines the PeerSync system boundary and the five actors that interact with it — Admin, Student, Professor, Researcher, and Industry Partner — along with their respective use cases and the include/extend relationships between them.



4.2 UI / Wireframe Design using Figma

The wireframes below were built directly from the SRS, with each core screen mapped to the functional requirements and user stories defined above — covering onboarding (Get Started, Sign Up, Log In), the Home feed and Discover/search screens, project and team creation, the team dashboard with resources and code sharing, in-app chat, the public profile and peer-rating flow, and the post-submission confirmation screen.

PeerSync Mobile App Design



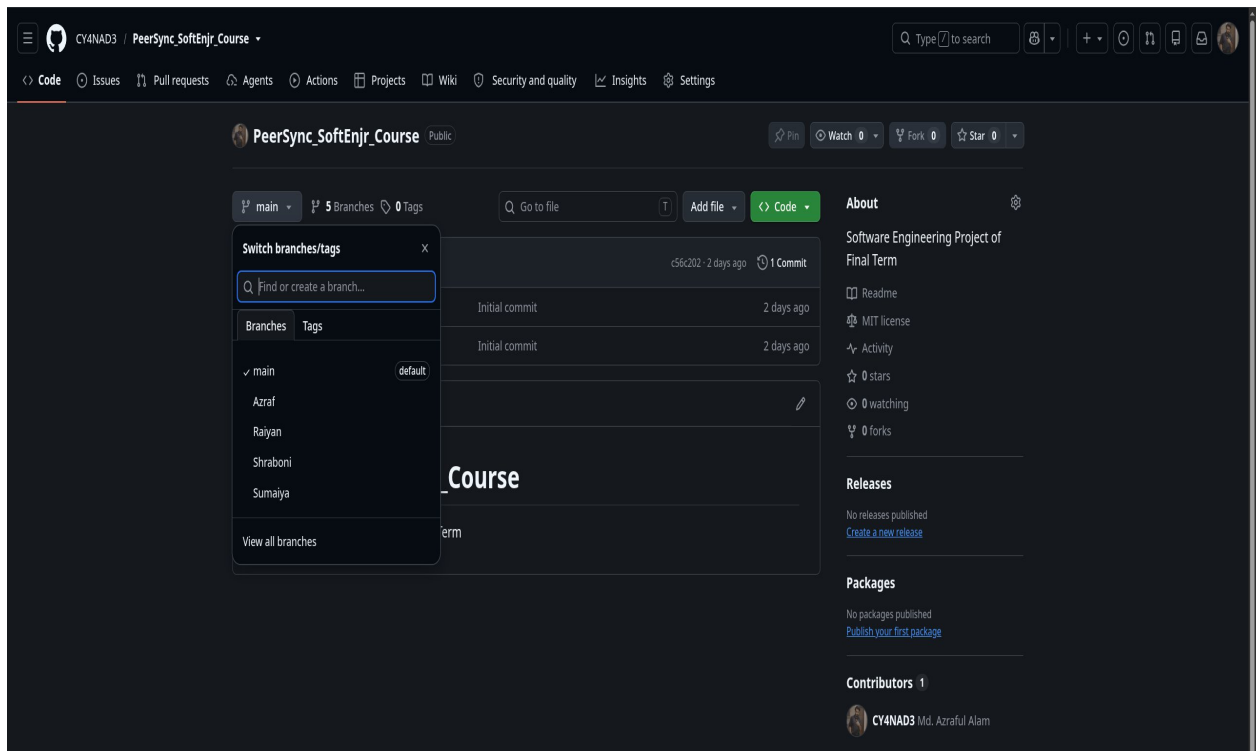
Figma prototype link:

<https://www.figma.com/design/cKzR72fFDUwHM8TV7zOMMv/Untitled?node-id=0-1&t=C4aLqJauIX2uy6CN-1>

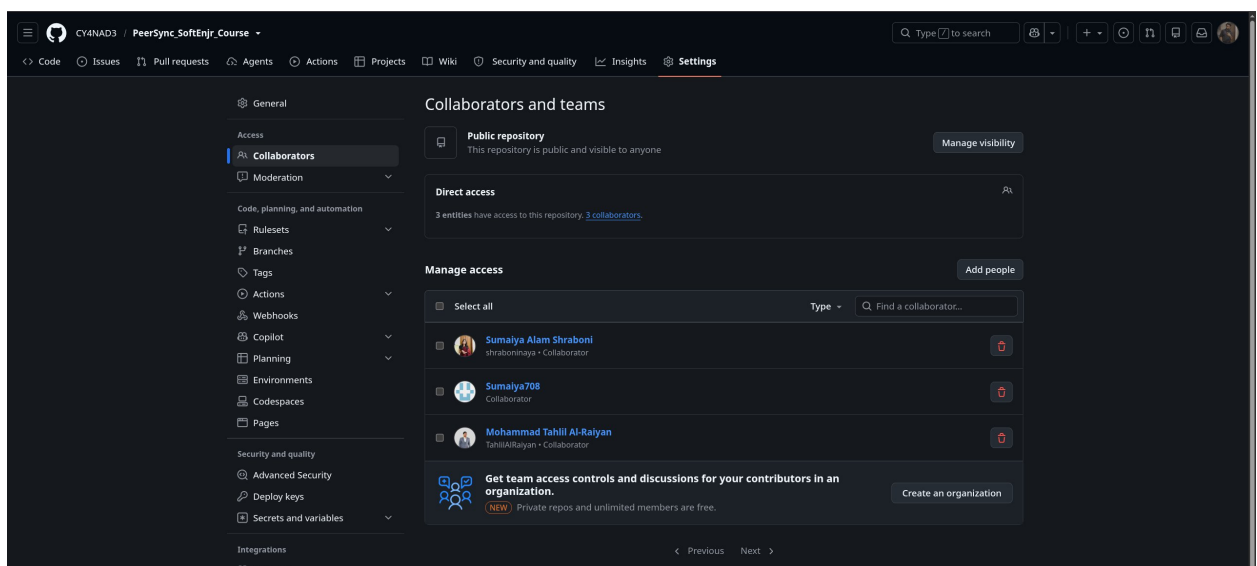
5. Git Workflow

GitHub repository link: https://github.com/CY4NAD3/PeerSync_SoftEnjr_Course.git

A central repository for the PeerSync project was created on GitHub (PeerSync_SoftEnjr_Course), with each team member added as a collaborator and working on their own dedicated branch named after them (Azraf, Raiyan, Shraboni, Sumaiya), branched off main.



GitHub repository showing member branches (main, Azraf, Raiyan, Shraboni, Sumaiya).



Repository collaborators and access settings.

Each member cloned the repository locally, created and checked out their own branch, and pushed it upstream to GitHub before adding files corresponding to their assigned Trello cards, staging them, committing with a descriptive message, and pushing the commit to their branch.

Raiyan

```
MINGW64;e/PeerSync_SoftEnjr_Course
raiya@_the_sowrdfish_ MINGW64 /e/PeerSync_SoftEnjr_Course
$ cd "E:\PeerSync_SoftEnjr_Course"

raiya@_the_sowrdfish_ MINGW64 /e/PeerSync_SoftEnjr_Course
$ git clone "https://github.com/CY4NAD3/PeerSync_SoftEnjr_Course.git"
Cloning into 'PeerSync_SoftEnjr_Course'...
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (4/4), done.

raiya@_the_sowrdfish_ MINGW64 /e/PeerSync_SoftEnjr_Course
$ |
```

```
raiya@_the_sowrdfish_ MINGW64 /e/PeerSync_SoftEnjr_Course/PeerSync_SoftEnjr_Course (main)
$ git branch Raiyan

raiya@_the_sowrdfish_ MINGW64 /e/PeerSync_SoftEnjr_Course/PeerSync_SoftEnjr_Course (main)
$ git checkout Raiyan
Switched to branch 'Raiyan'

raiya@_the_sowrdfish_ MINGW64 /e/PeerSync_SoftEnjr_Course/PeerSync_SoftEnjr_Course (Raiyan)
$ git push -u origin Raiyan
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'Raiyan' on GitHub by visiting:
remote:   https://github.com/CY4NAD3/PeerSync_SoftEnjr_Course/pull/new/Raiya
remote:
To https://github.com/CY4NAD3/PeerSync_SoftEnjr_Course.git
 * [new branch]      Raiyan -> Raiyan
branch 'Raiyan' set up to track 'origin/Raiyan'.

raiya@_the_sowrdfish_ MINGW64 /e/PeerSync_SoftEnjr_Course/PeerSync_SoftEnjr_Course (Raiyan)
$
```

Branch creation, checkout, and upstream push.

```

raiya@_the_sowrdfish_ MINGW64 /e/PeerSync_SoftEnjr_Course/PeerSync_SoftEnjr_Course (main)
$ git branch Raiyan

raiya@_the_sowrdfish_ MINGW64 /e/PeerSync_SoftEnjr_Course/PeerSync_SoftEnjr_Course (main)
$ git checkout Raiyan
Switched to branch 'Raiyan'

raiya@_the_sowrdfish_ MINGW64 /e/PeerSync_SoftEnjr_Course/PeerSync_SoftEnjr_Course (Raiyan)
$ git push -u origin Raiyan
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'Raiyan' on GitHub by visiting:
remote:   https://github.com/CY4NAD3/PeerSync_SoftEnjr_Course/pull/new/Raiya
remote:
To https://github.com/CY4NAD3/PeerSync_SoftEnjr_Course.git
 * [new branch]      Raiyan -> Raiyan
branch 'Raiyan' set up to track 'origin/Raiyan'.

raiya@_the_sowrdfish_ MINGW64 /e/PeerSync_SoftEnjr_Course/PeerSync_SoftEnjr_Course (Raiyan)
$ git add SWE_FR_NFR_Group-05_Sec-E.docx

raiya@_the_sowrdfish_ MINGW64 /e/PeerSync_SoftEnjr_Course/PeerSync_SoftEnjr_Course (Raiyan)
$ git commit -m "NFR and FR doc uploaded"
[Raiyan 2ae5c4a] NFR and FR doc uploaded
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 SWE_FR_NFR_Group-05_Sec-E.docx

raiya@_the_sowrdfish_ MINGW64 /e/PeerSync_SoftEnjr_Course/PeerSync_SoftEnjr_Course (Raiyan)
$ git push origin Raiyan
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 19.75 KiB | 6.58 MiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/CY4NAD3/PeerSync_SoftEnjr_Course.git
 c56c202..2ae5c4a  Raiyan -> Raiyan

raiya@_the_sowrdfish_ MINGW64 /e/PeerSync_SoftEnjr_Course/PeerSync_SoftEnjr_Course (Raiyan)
$

```

Adding, committing, and pushing the FR/NFR document.

Sumaiya

```
MINGW64:/c/PeerSync_SoftEnjr_Course
Admin@hp MINGW64 /c/PeerSync_SoftEnjr_Course
$ git clone "https://github.com/CY4NAD3/PeerSync_SoftEnjr_Course"
Cloning into 'PeerSync_SoftEnjr_Course'...
remote: Enumerating objects: 10, done.
remote: Counting objects: 100% (10/10), done.
remote: Compressing objects: 100% (9/9), done.
remote: Total 10 (delta 3), reused 5 (delta 1), pack-reused 0 (from 0)
Receiving objects: 100% (10/10), 28.11 KiB | 4.69 MiB/s, done.
Resolving deltas: 100% (3/3), done.

Admin@hp MINGW64 /c/PeerSync_SoftEnjr_Course
$
```

```
MINGW64:/c/PeerSync_SoftEnjr_Course/PeerSync_SoftEnjr_Course
Admin@hp MINGW64 /c/PeerSync_SoftEnjr_Course/PeerSync_SoftEnjr_Course (main)
$ git branch Sumaiya

Admin@hp MINGW64 /c/PeerSync_SoftEnjr_Course/PeerSync_SoftEnjr_Course (main)
$ git checkout Sumaiya
Switched to branch 'Sumaiya'

Admin@hp MINGW64 /c/PeerSync_SoftEnjr_Course/PeerSync_SoftEnjr_Course (Sumaiya)
$ git push -u origin Sumaiya
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'Sumaiya' on GitHub by visiting:
remote:   https://github.com/CY4NAD3/PeerSync_SoftEnjr_Course/pull/new/Sumaiy
a
remote:
To https://github.com/CY4NAD3/PeerSync_SoftEnjr_Course
 * [new branch]      Sumaiya -> Sumaiya
branch 'Sumaiya' set up to track 'origin/Sumaiya'.

Admin@hp MINGW64 /c/PeerSync_SoftEnjr_Course/PeerSync_SoftEnjr_Course (Sumaiya)
$ git add .

Admin@hp MINGW64 /c/PeerSync_SoftEnjr_Course/PeerSync_SoftEnjr_Course (Sumaiya)
$ git commit -m "Process Model and Scopes Features file uploaded"
[Sumaiya c474869] Process Model and Scopes Features file uploaded
2 files changed, 0 insertions(+), 0 deletions(-)
create mode 100644 PeerSync_Scopes and Features.docx
create mode 100644 SE_SelectionOfProcessModel.docx

Admin@hp MINGW64 /c/PeerSync_SoftEnjr_Course/PeerSync_SoftEnjr_Course (Sumaiya)
$ git push origin Sumaiya
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 163.52 KiB | 16.35 MiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/CY4NAD3/PeerSync_SoftEnjr_Course
 c56c202..c474869 Sumaiya -> Sumaiya

Admin@hp MINGW64 /c/PeerSync_SoftEnjr_Course/PeerSync_SoftEnjr_Course (Sumaiya)
$
```

Branch creation, checkout, upstream push, and commit of the Process Model and Scopes & Features documents.

Azraf

```
PeerSync_SoftEnjr_Course : fish — Konsole
New Tab Split View
PeerSync_SoftEnjr_Course : fish PeerSync_SoftEnjr_Course : fish

/run/media/azraf/SSD samsung
> git clone "https://github.com/CY4NAD3/PeerSync_SoftEnjr_Course.git"
Cloning into 'PeerSync_SoftEnjr_Course'...
remote: Enumerating objects: 7, done.
remote: Counting objects: 100% (7/7), done.
remote: Compressing objects: 100% (7/7), done.
remote: Total 7 (delta 1), reused 3 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (7/7), done.
Resolving deltas: 100% (1/1), done.

/run/media/azraf/SSD samsung
> ls
drwxrwxrwx - root 9 Mar 01:48 - !Competitive__programming
drwxrwxrwx - root 4 Sep 2025 - !C++
drwxrwxrwx - root 19 May 17:00 - !CSharp
drwxrwxrwx - root 8 Mar 21:05 - !DSA
drwxrwxrwx - root 13 Mar 14:36 - !Harvard_CS50
drwxrwxrwx - root 8 Mar 12:49 - !IDB
drwxrwxrwx - root 4 Sep 2025 - !Java
drwxrwxrwx - root 16 May 2025 - $RECYCLE.BIN
drwxrwxrwx - root 28 Aug 16:53 - -ComputerGraphics
drwxrwxrwx - root 21 Jul 14:51 - Docker
drwxrwxrwx - root 17 Aug 2025 - EALibrary
drwxrwxrwx - root 21 Feb 02:51 - EptLibrary
drwxrwxrwx - root 28 Aug 19:49 - found.000
drwxrwxrwx - root 9 May 10:47 - 'Lossless Scaling'
drwxrwxrwx - root 15 Mar 23:18 - 'Neon Night Explorer V-1.3'
drwxrwxrwx - root 8 May 23:12 - 'Neverness To Everness'
drwxrwxrwx - root 8 Mar 12:22 - Oracle_soft
drwxrwxrwx - root 28 Aug 23:39 - PeerSync_SoftEnjr_Course
drwxrwxrwx - root 7 Mar 16:32 - 'Program Files'
drwxrwxrwx - root 20 May 23:12 - 'Riot Games'
drwxrwxrwx - root 28 Aug 22:35 - SteamLibrary
drwxrwxrwx - root 27 Jan 22:19 - StoveLibrary
drwxrwxrwx - root 28 Aug 15:13 - 'System Volume Information'
drwxrwxrwx - root 16 May 2025 - *VBOS os
drwxrwxrwx - root 7 Mar 16:32 - WindowsApps
```

Cloning the repository.

```
/run/media/azraf/SSD samsung/PeerSync_SoftEnjr_Course main
> git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

/run/media/azraf/SSD samsung/PeerSync_SoftEnjr_Course main
> git branch Azraf

/run/media/azraf/SSD samsung/PeerSync_SoftEnjr_Course main
> git checkout Azraf
Switched to branch 'Azraf'
```

Checking status, then creating and checking out the Azraf branch.

```
PeerSync_SoftEnjr_Course : fish — Konsole
New Tab Split View
PeerSync_SoftEnjr_Course : fish PeerSync_SoftEnjr_Course : fish

/run/media/azraf/SSD samsung/PeerSync_SoftEnjr_Course Azraf* 28s
> git push -u origin Azraf
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'Azraf' on GitHub by visiting:
remote: https://github.com/CY4NAD3/PeerSync_SoftEnjr_Course/pull/new/Azraf
remote:
To https://github.com/CY4NAD3/PeerSync_SoftEnjr_Course.git
 * [new branch]      Azraf -> Azraf
branch 'Azraf' set up to track 'origin/Azraf'.

/run/media/azraf/SSD samsung/PeerSync_SoftEnjr_Course Azraf*
> git add
Nothing specified, nothing added.
hint: Maybe you wanted to say 'git add .'
hint: Disable this message with "git config set advice.addEmptyPaths false"

/run/media/azraf/SSD samsung/PeerSync_SoftEnjr_Course Azraf*
> git add .

/run/media/azraf/SSD samsung/PeerSync_SoftEnjr_Course Azraf*
> git commit -m "Project Proposal and UseCase Diagram commit"
[Azraf f03cbbe] Project Proposal and UseCase Diagram commit
2 files changed, 6681 insertions(+)
create mode 100644 Project_Proposal_Group_05_2026.docx
create mode 100644 UseCase_Diagram.pdf

/run/media/azraf/SSD samsung/PeerSync_SoftEnjr_Course Azraf*
> git push origin Azraf
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 383.31 KiB | 34.85 MiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/CY4NAD3/PeerSync_SoftEnjr_Course.git
 c56c202..f03cbbe Azraf -> Azraf
```

Committing and pushing the Project Proposal and Use Case Diagram.

Shraboni

```
MINGW64/c:/Users/user/OneDrive/Desktop/Shraboni/PeerSync_SoftEnjr_Course

user@SHRABONI MINGW64 ~/OneDrive/Desktop/Shraboni/PeerSync_SoftEnjr_Course (main)
$ git checkout Shraboni
Switched to branch 'Shraboni'
Your branch is up to date with 'origin/Shraboni'.

user@SHRABONI MINGW64 ~/OneDrive/Desktop/Shraboni/PeerSync_SoftEnjr_Course (Shraboni)
$ git add .

user@SHRABONI MINGW64 ~/OneDrive/Desktop/Shraboni/PeerSync_SoftEnjr_Course (Shraboni)
$ git commit -m "Shraboni push"
On branch Shraboni
Your branch is up to date with 'origin/Shraboni'.

nothing to commit, working tree clean

user@SHRABONI MINGW64 ~/OneDrive/Desktop/Shraboni/PeerSync_SoftEnjr_Course (Shraboni)
$ git push origin -u Shraboni
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 12.99 MiB | 3.16 MiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'Shraboni' on GitHub by visiting:
remote:   https://github.com/CY4NAD3/PeerSync_SoftEnjr_Course/pull/new/Shraboni
remote:
To https://github.com/CY4NAD3/PeerSync_SoftEnjr_Course.git
 * [new branch]      Shraboni -> Shraboni
branch 'Shraboni' set up to track 'origin/Shraboni'.

user@SHRABONI MINGW64 ~/OneDrive/Desktop/Shraboni/PeerSync_SoftEnjr_Course (Shraboni)
$
```

Checkout, add, commit, and upstream push Figma SS & User Stories document file on the Shraboni branch.

Git Merge

```
PeerSync_SoftEnjr_Course: fish — Konsole

New Tab Split View Copy Paste Find

/run/media/azraf/SSD samsung/PeerSync_SoftEnjr_Course main
$ git pull origin main
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 3.09 KiB | 452.00 KiB/s, done.
From https://github.com/CY4NAD3/PeerSync_SoftEnjr_Course
+ branch main -> FETCH_HEAD
c56c202..85409b8 main -> origin/main
Updating c56c202..85409b8
Fast-forward
 README.md | 68 ++++++
 1 file changed, 66 insertions(+), 2 deletions(-)

/run/media/azraf/SSD samsung/PeerSync_SoftEnjr_Course main
$ ls
drwxrwxrwx 1 azraf 14 Sep 00:42 .git
-rwxrwxrwx 1.1k azraf 28 Aug 23:39 LICENSE
-rwxrwxrwx 5.4k azraf 14 Sep 00:36 PeerSync_SE_Project_Report(4).docx
-rwxrwxrwx 8.2k azraf 14 Sep 00:36 PeerSync_SE_Project_Report(4).pdf
-rwxrwxrwx 4.1k azraf 14 Sep 00:42 README.md

/run/media/azraf/SSD samsung/PeerSync_SoftEnjr_Course main
$ git add .

/run/media/azraf/SSD samsung/PeerSync_SoftEnjr_Course main
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   PeerSync_SE_Project_Report(4).docx
        new file:   PeerSync_SE_Project_Report(4).pdf

/run/media/azraf/SSD samsung/PeerSync_SoftEnjr_Course main
$ git commit -m "Project report .docx and .pdf upload"
[main b9f8642] Project report .docx and .pdf upload
2 files changed, 0 insertions(+), 0 deletions(-)
create mode 100644 PeerSync_SE_Project_Report(4).docx
create mode 100644 PeerSync_SE_Project_Report(4).pdf

/run/media/azraf/SSD samsung/PeerSync_SoftEnjr_Course main
$ git push origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 7.30 MiB | 5.53 MiB/s, done.
Total 4 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/CY4NAD3/PeerSync_SoftEnjr_Course.git
 85409b8..b9f8642 main -> main

/run/media/azraf/SSD samsung/PeerSync_SoftEnjr_Course main
```

Pulling latest main, listing files, then staging, committing, and pushing the updated Project Report (.docx and .pdf).

```
/run/media/azraf/SSD samsung/PeerSync_SoftEnjr_Course main
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

/run/media/azraf/SSD samsung/PeerSync_SoftEnjr_Course main
$ git fetch origin
remote: Enumerating objects: 13, done.
remote: Counting objects: 100% (13/13), done.
remote: Compressing objects: 100% (10/10), done.
remote: Total 11 (delta 1), reused 11 (delta 1), pack-reused 0 (from 0)
Unpacking objects: 100% (11/11), 13.16 MiB | 8.17 MiB/s, done.
From https://github.com/CY4NAD3/PeerSync_SoftEnjr_Course
 2ae5c4a..e9bcf51 Ralyan -> origin/Ralyan
+ [new branch] Shraboni -> origin/Shraboni
+ [new branch] Sumaiya -> origin/Sumaiya

/run/media/azraf/SSD samsung/PeerSync_SoftEnjr_Course main
$ git branch -a
Azraf
* main
remotes/origin/Azraf
remotes/origin/HEAD -> origin/main
remotes/origin/Ralyan
remotes/origin/Shraboni
remotes/origin/Sumaiya
remotes/origin/main
```

Checking status (clean), fetching updates from origin, then listing all local and remote branches.

```
PeerSyncSoftEnjr_Course : fish — Konsole

/run/media/azraf/SSD samsung/PeerSyncSoftEnjr_Course main
> git merge origin/Azraf
hint: Waiting for your editor to close the file... error: cannot run vi: No such file or directory
error: unable to start editor 'vi'
Not committing merge; use 'git commit' to complete the merge.

/run/media/azraf/SSD samsung/PeerSyncSoftEnjr_Course main
> git config --global core.editor "nano"
git: 'config' is not a git command. See 'git --help'.

The most similar command is
    config

/run/media/azraf/SSD samsung/PeerSyncSoftEnjr_Course main
> git config --global core.editor "nano"

/run/media/azraf/SSD samsung/PeerSyncSoftEnjr_Course main
> git commit
[main 1119224] Merge remote-tracking branch 'origin/Azraf'

/run/media/azraf/SSD samsung/PeerSyncSoftEnjr_Course main : 48s
> git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 389 bytes | 389.00 KiB/s, done.
Total 2 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/CY4NAD3/PeerSyncSoftEnjr_Course.git
   b9f8642..1119224  main -> main

/run/media/azraf/SSD samsung/PeerSyncSoftEnjr_Course main
> git merge origin/Raiyan
Merge made by the 'ort' strategy.
 SWE_FR_NFR_Group-05_Sec-E.docx | Bin 0 -> 22978 bytes
 Timeline_Chart_Group_05_2026.docx | Bin 0 -> 16966 bytes
 2 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 SWE_FR_NFR_Group-05_Sec-E.docx
 create mode 100644 Timeline_Chart_Group_05_2026.docx

/run/media/azraf/SSD samsung/PeerSyncSoftEnjr_Course main : 55s
> git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 401 bytes | 401.00 KiB/s, done.
Total 2 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/CY4NAD3/PeerSyncSoftEnjr_Course.git
   1119224..351110f  main -> main

/run/media/azraf/SSD samsung/PeerSyncSoftEnjr_Course main
> git merge origin/Sumaiya
Merge made by the 'ort' strategy.
 PeerSync_Scopes_and_Features.docx | Bin 0 -> 10975 bytes
 SE_SelectionOfProcessModel.docx | Bin 0 -> 159300 bytes
 2 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 PeerSync_Scopes_and_Features.docx
 create mode 100644 SE_SelectionOfProcessModel.docx
```

Merging Azraf (fixing the missing-editor error, setting nano, then committing and pushing), followed by merging and pushing Raiyan, then merging Sumaiya.

```
PeerSyncSoftEnjr_Course : fish — Konsole

Total 2 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/CY4NAD3/PeerSyncSoftEnjr_Course.git
   1119224..351110f  main -> main

/run/media/azraf/SSD samsung/PeerSyncSoftEnjr_Course main
> git merge origin/Sumaiya
Merge made by the 'ort' strategy.
 PeerSync_Scopes_and_Features.docx | Bin 0 -> 10975 bytes
 SE_SelectionOfProcessModel.docx | Bin 0 -> 159300 bytes
 2 files changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 PeerSync_Scopes_and_Features.docx
 create mode 100644 SE_SelectionOfProcessModel.docx

/run/media/azraf/SSD samsung/PeerSyncSoftEnjr_Course main :
> git push origin HEAD
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 392 bytes | 392.00 KiB/s, done.
Total 2 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/CY4NAD3/PeerSyncSoftEnjr_Course.git
   351110f..12f1685  HEAD -> main

/run/media/azraf/SSD samsung/PeerSyncSoftEnjr_Course main
> git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

/run/media/azraf/SSD samsung/PeerSyncSoftEnjr_Course main
> git merge origin/Shraboni
git push origin HEAD
Merge made by the 'ort' strategy.
 Group 254 (1).pdf | 97116 -----
 PeerSync_User_Stories (1).docx | Bin 0 -> 20762 bytes
 2 files changed, 97116 insertions(+)
 create mode 100644 Group 254 (1).pdf
 create mode 100644 PeerSync_User_Stories (1).docx
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 12 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 379 bytes | 379.00 KiB/s, done.
Total 2 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/CY4NAD3/PeerSyncSoftEnjr_Course.git
   12f1685..1b2b4d6  HEAD -> main

/run/media/azraf/SSD samsung/PeerSyncSoftEnjr_Course main : 7s
> git status
On branch main
Your branch is up to date with 'origin/main'.

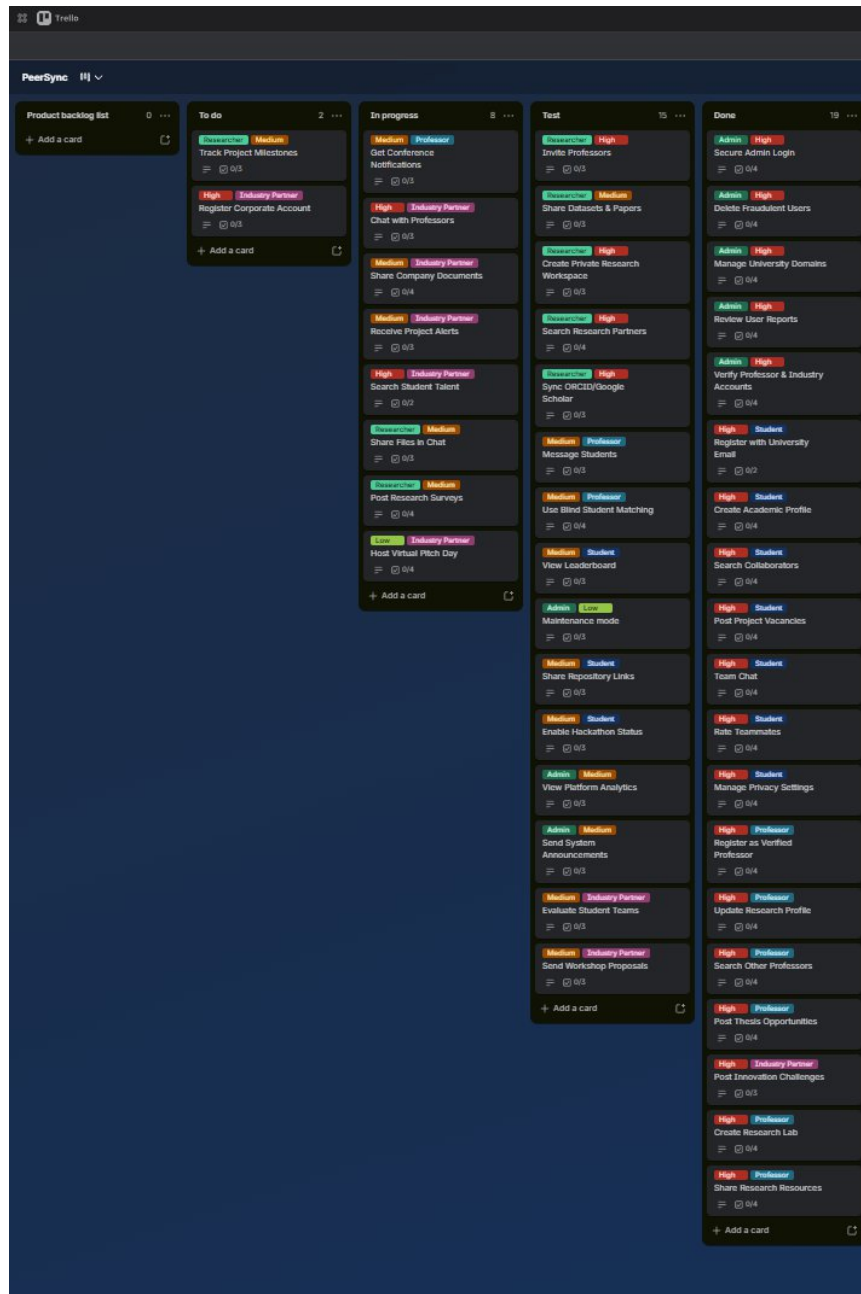
nothing to commit, working tree clean
```

Pushing the Sumaiya merge, confirming a clean status, then merging and pushing Shraboni — completing all four branch merges into main.

Final Trello Board

Trello board link:

<https://trello.com/invite/b/6a53471ef5178fa45525a0af/ATTI680e9780a3c4c5a5c70dd88d83da2fb6795ADC31/peersync>



6. Conclusion

PeerSync addresses a problem that most academic collaboration tools overlook: finding the right teammate, mentor, or research partner is still largely left to chance or existing social circles, even though the underlying need for skill-based, verified collaboration spans students, professors, researchers, and industry partners alike. Building on the original AIUB-only matching concept, this semester's work expanded PeerSync into a multi-university platform, guided by a formal Scrum process across nine phases from planning through review and delivery. The Software Requirements Specification defines the platform's scope through 44 functional requirements, 15 non-functional requirements, and 44 user stories spanning five distinct roles, ensuring that every feature traces back to a real stakeholder need rather than an assumed one. The use case diagram and Figma wireframes translate those requirements into a concrete system boundary and user interface, covering onboarding, discovery, project collaboration, messaging, and reputation-building end to end. What remains is to close the loop between design and implementation: completing the GitHub-based development workflow and carrying this SRS and design foundation into a working, testable version of PeerSync in the semesters ahead.